

PRACTICAL WEEK 5

Task:-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source code:-

```
GNU nano 8.7.1 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define SIZE 1000

int main()
{
    int pipefd[2];
    int data[SIZE];
    pid_t pid;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    clock_gettime(CLOCK_MONOTONIC, &start);
    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }
    if (pid > 0)
```

```
GNU nano 8.7.1 producer_consumer.c *
    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_nsec - start.tv_nsec) / 1000.0;

    printf("\nProducer: Generated %d integers.\n", SIZE);
    printf("Communication time: %.2f microseconds\n", time_taken);
    printf("Data transferred: %lu bytes\n", sizeof(data));
    printf("Communication efficiency: %.2f bytes/microsecond\n",
        sizeof(data) / time_taken);
}
else
{
    // Child - Consumer
    close(pipefd[1]);

    int received[SIZE];

    read(pipefd[0], received, sizeof(received));

    close(pipefd[0]);

    printf("Consumer: Received %d integers.\n", SIZE);
    printf("First value: %d\n", received[0]);
    printf("Last value: %d\n", received[SIZE - 1]);
}

return 0;
}
```

OUTPUT:-

```

anuan@Anushka:~$ nano producer_consumer.c
anuan@Anushka:~$ gcc producer_consumer.c -o producer_consumer
anuan@Anushka:~$ ./producer_consumer
Consumer: Received 1000 integers.
First value: 1
Last value: 1000

Producer: Generated 1000 integers.
Communication time: 550.64 microseconds
Data transferred: 4000 bytes
Communication efficiency: 7.26 bytes/microsecond
anuan@Anushka:~$ |

```

Observation

The parent process acts as the **producer** and sends data through an anonymous pipe. The child process acts as the **consumer** and reads the data from the pipe. The communication time and data transfer rate are measured to evaluate the efficiency of inter-process communication.

Task:-2

Develop a program that executes the equivalent of the shell command:

`ls -l | grep ".c"`

using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Source code:-

```

GNU nano 8.7.1 pipe_exec.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    // First child: ls -l
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);
    }
}

```

```
if (pid2 < 0)
{
    perror("fork");
    exit(1);
}

if (pid2 == 0)
{
    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);
    close(pipefd[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

// Parent closes pipe
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nCommand executed: ls -l | grep \".c\"\n");

return 0;
}
```

OUTPUT

```

anuan@Anushka:~$ nano pipe_exec.c
anuan@Anushka:~$ gcc pipe_exec.c -o pipe_exec
anuan@Anushka:~$ ./pipe_exec
drwxr-xr-x 11 anuan anuan 4096 Aug 7 03:59 Practical
-rwxr-xr-x 1 anuan anuan 16400 Aug 8 03:24 command_executor
-rw-r--r-- 1 anuan anuan 1143 Aug 8 03:24 command_executor.c
-rw-r--r-- 1 anuan anuan 1988 Aug 25 04:55 double_quotes.c
-rw-r--r-- 1 anuan anuan 1990 Aug 25 04:57 double_quotes.c.save
-rwxr-xr-x 1 anuan anuan 16216 Aug 7 05:27 file_copy
-rw-r--r-- 1 anuan anuan 1113 Aug 7 05:27 file_copy.c
-rw-r--r-- 1 anuan anuan 0 Aug 7 05:25 file_copy.c.save
-rwxr-xr-x 1 anuan anuan 16080 Aug 5 03:06 fork_exec
-rw-r--r-- 1 anuan anuan 516 Aug 5 03:01 fork_exec.c
-rw-r--r-- 1 anuan anuan 517 Aug 5 03:04 fork_exec.c.save
-rw-r--r-- 1 anuan anuan 1299 Aug 5 03:05 fork_exec.r
-rw-r--r-- 1 anuan anuan 728 Aug 8 03:20 hardware_os.c
-rw-r--r-- 1 anuan anuan 151 Aug 17 04:11 hello.c
-rw-r--r-- 1 anuan anuan 1803 Aug 7 05:37 keyboard_input.c
-rw-r--r-- 1 anuan anuan 4239 Aug 11 06:43 lex_parser.c
-rw-r--r-- 1 anuan anuan 559 Aug 17 04:04 main_loop.c
-rw-r--r-- 1 anuan anuan 1109 Aug 24 09:19 p.c
-rw-r--r-- 1 anuan anuan 8864 Aug 11 06:34 parser.c
-rw-r--r-- 1 anuan anuan 8864 Aug 18 06:54 parser_lexer.c
-rw-r--r-- 1 anuan anuan 8533 Aug 11 06:42 parser_lexer.c.save
-rw-r--r-- 1 anuan anuan 680 Aug 24 09:19 pc.c.save
-rw-r--r-- 1 anuan anuan 1113 Aug 24 09:39 pd.c
-rw-r--r-- 1 anuan anuan 1301 Aug 24 09:51 pi.c.save
-rwxr-xr-x 1 anuan anuan 16344 Aug 29 03:14 pipe_exec
-rw-r--r-- 1 anuan anuan 1164 Aug 29 03:14 pipe_exec.c
-rw-r--r-- 1 anuan anuan 1161 Aug 24 09:50 pk.c
-rw-r--r-- 1 anuan anuan 685 Aug 24 09:47 pl.c
-rw-r--r-- 1 anuan anuan 800 Aug 24 09:33 pr.c.save
-rwxr-xr-x 1 anuan anuan 16296 Aug 17 04:08 process
-rw-r--r-- 1 anuan anuan 81 Aug 4 05:10 process.
-rw-r--r-- 1 anuan anuan 883 Aug 17 04:08 process.c
-rw-r--r-- 1 anuan anuan 797 Aug 4 05:35 process.c.save
-rw-r--r-- 1 anuan anuan 1497 Aug 24 08:59 process.c.save.1
-rwxr-xr-x 1 anuan anuan 16296 Aug 8 02:52 process_states
-rw-r--r-- 1 anuan anuan 1376 Aug 8 02:52 process_states.c
-rwxr-xr-x 1 anuan anuan 16400 Aug 29 03:10 producer_consumer
-rw-r--r-- 1 anuan anuan 1564 Aug 29 03:10 producer_consumer.c
-rw-r--r-- 1 anuan anuan 1388 Aug 24 09:04 product.c.save
-rw-r--r-- 1 anuan anuan 676 Aug 6 05:22 program.c
-rw-r--r-- 1 anuan anuan 965 Aug 24 09:25 ps.c
-rw-r--r-- 1 anuan anuan 1137 Aug 24 09:45 pt.c
-rw-r--r-- 1 anuan anuan 872 Aug 25 04:47 single_quotes.c
-rw-r--r-- 1 anuan anuan 780 Aug 8 03:04 state_monitor.c
-rw-r--r-- 1 anuan anuan 726 Aug 20 05:17 task2.c
-rw-r--r-- 1 anuan anuan 723 Aug 20 05:15 task2.c.save
-rw-r--r-- 1 anuan anuan 1137 Aug 20 05:10 wait_waitpid.c
-rw-r--r-- 1 anuan anuan 987 Aug 18 06:52 waitpid_demo.c

Command executed: ls -l | grep ".c"
anuan@Anushka:~$

```

Observation

The program successfully implements the shell pipeline `ls -l | grep ".c"` using Linux system calls. The first child executes `ls -l` and sends its output through the pipe. The second child receives the output through the pipe and executes `grep ".c"` to display C-related files